Система навыков

Навыки — это документы со знаниями, которые агент может загрузить при необходимости. Они следуют принципу **прогрессивного раскрытия** для минимизации использования токенов и совместимы с открытым стандартом agentskills.io.

Все навыки хранятся в `~/.hermes/skills/` — основном каталоге и достоверном источнике информации. При новой установке связанные навыки копируются из репозитория. Сюда же попадают навыки, установленные в Hub и созданные агентом. Агент может изменить или удалить любой навык.

Вы также можете указать Hermes на **внешние каталоги навыков** — дополнительные папки, которые сканируются вместе с локальной. См. Внешние каталоги навыков ниже.

См. также:

- Пакетный каталог навыков
- Официальный дополнительный каталог навыков

Начинать с чистого листа

По умолчанию в каждый профиль добавляется объединенный каталог навыков, и каждый `hermes update` добавляет новые объединенные навыки. Если вам нужен профиль с **без объединенных навыков**, который остается пустым после обновлений, у вас есть два варианта:

Во время установки (применимо к профилю `~/.hermes` по умолчанию):

```
curl -fsSL https://hermes-agent.nousresearch.com/install.sh | bash -s -- --
no-skills
```

При создании профиля (именованные профили):

```
hermes profile create research --no-skills
```

В уже установленном профиле (по умолчанию или с именем) его можно включить во время выполнения:

```
hermes skills opt-out            # stop future seeding — nothing on disk is touched
hermes skills opt-out --remove   # also delete UNMODIFIED bundled skills (confirms first)
hermes skills opt-in --sync      # undo: remove the marker and re-seed now
```

Все три пути записывают маркер `.no-bundled-skills` в каталог профиля. Пока маркер присутствует, установщик, `hermes update` и любая синхронизация навыков пропускают установку связанных навыков для этого профиля. Чтобы снова включить эту функцию, удалите маркер (или запустите `hermes skills opt-in`).

> **БЕЗОПАСНЫЙ ПО УМОЛЧАНИЮ**
>
> `hermes skills opt-out` Эта команда останавливает *будущее* распространение, но никогда не удаляет то, что уже есть на диске. Необязательный флаг `--remove` удаляет связанные навыки **только** в том случае, если они не были изменены (идентичны по байтам установленной версии Hermes). Навыки, которые вы редактировали, навыки, установленные из хаба, и навыки, которые вы написали сами, сохраняются.

Использование навыков

Каждый установленный навык автоматически доступен в виде команды со слэшем:

```
# In the CLI or any messaging platform:
/gif-search funny cats
/axolotl help me fine-tune Llama 3 on my dataset
/github-pr-workflow create a PR for the auth refactor
/plan design a rollout for migrating our auth provider

# Just the skill name loads it and lets the agent ask what you need:
/excalidraw
```

Объединение нескольких навыков в одной команде

Вы можете использовать несколько навыков в одном сообщении, объединив команды с косой чертой в начале — загружается каждый ведущий токен `/skill` (до 5), а остальное становится вашей инструкцией:

```
/github-pr-workflow /test-driven-development fix issue #123 and open a PR
```

Парсинг останавливается на первом токене, который не является установленным навыком, поэтому аргументы, начинающиеся с `/` (например, пути к файлам), никогда не игнорируются:

```
/ocr-and-documents /tmp/scan.pdf extract the tables   # loads one skill;
/tmp/scan.pdf is the argument
```

Для комбинаций, которые вы используете часто, лучше выбрать набор навыков — тот же эффект, но с одной короткой командой.

Хорошо подходит для этого упакованный навык `plan` . При выполнении `/plan [request]` загружаются инструкции навыка, которые указывают Hermes на необходимость проверки контекста, составления плана реализации в формате Markdown вместо выполнения задачи и сохранения результата в папке `.hermes/plans/` относительно активного рабочего каталога рабочей области/бэкенда.

Вы также можете взаимодействовать с навыками в ходе обычного разговора:

```
hermes chat --toolsets skills -q "What skills do you have?"
hermes chat --toolsets skills -q "Show me the axolotl skill"
```

Обучение навыкам по источникам (`/learn`)

`/learn` Это быстрый способ превратить то, что вам уже известно, или набор справочных материалов, в многократно используемый навык, не прописывая `SKILL.md`. Это универсальный подход: укажите *все, что вы можете описать*, и агент соберет материал с помощью уже имеющихся у него инструментов, а затем создаст навык, соответствующий домашним стандартам (описание длиной ≤60 символов,

стандартный порядок разделов, оформление с помощью инструмента Hermes, отсутствие изобретенных команд).

```
# A local SDK or doc directory — read with read_file / search_files
/learn the REST client in ~/projects/acme-sdk, focus on auth + pagination

# An online doc page — fetched with web_extract
/learn https://docs.example.com/api/quickstart

# The workflow you just walked the agent through in this conversation
/learn how I just deployed the staging server

# Pasted notes / a described procedure
/learn filing an expense: open the portal, New > Expense, attach the
receipt, submit

# A whole book, paper stack, or large docs corpus — becomes a knowledge-
base skill
/learn ~/books/designing-data-intensive-applications.pdf
```

Large sources become knowledge-base skills

When the source is a book, a stack of papers, a spec, or a large docs folder, the agent doesn't cram it into one file or reduce it to a lossy summary. Instead it authors an **expansive knowledge-base skill**: a lean `SKILL.md` carrying the source's core mental models plus an index, with one distilled file per chapter or topic under `references/` (plus a glossary or cheatsheet when the source earns them). Reference files cost nothing until a question needs one — the agent loads them on demand with `skill_view`, so query cost stays proportional to the answer, not the source. Re-running `/learn` with new material on the same topic folds it into the existing skill rather than creating a duplicate.

The distillation synthesizes structure — frameworks, definitions, decision rules, anti-patterns — and never reproduces passages of the source text.

Because the live agent does the sourcing, `/learn` works the same in the CLI, the messaging gateway, the TUI, and the dashboard — and on any terminal backend (local, Docker, remote), since there is no separate ingestion engine. In the **dashboard**, the Skills page has a **Learn a skill** button that opens a panel with a directory field, a URL field, and an open-ended text box; it composes a `/learn` request and runs it in chat.

There is no model-tool footprint: `/learn` builds a standards-guided prompt and hands it to the agent as a normal turn. The agent saves the result with the `skill_manage` tool, so the

write-approval gate applies if you have it on.

Progressive Disclosure

Skills use a token-efficient loading pattern:

```
Level 0: skills_list()           → [{name, description, category}, ...]   (~3
Level 1: skill_view(name)        → Full content + metadata       (varies)
Level 2: skill_view(name, path)  → Specific reference file       (varies)
```

The agent only loads the full skill content when it actually needs it.

SKILL.md Format

```yaml
---
name: my-skill
description: Brief description of what this skill does
version: 1.0.0
platforms: [macos, linux]     # Optional — restrict to specific OS
platforms
metadata:
  hermes:
    tags: [python, automation]
    category: devops
    fallback_for_toolsets: [web]    # Optional — conditional activation
(see below)
    requires_toolsets: [terminal]   # Optional — conditional activation
(see below)
    config:                          # Optional — config.yaml settings
      - key: my.setting
        description: "What this controls"
        default: "value"
        prompt: "Prompt for setup"
---

# Skill Title

## When to Use
Trigger conditions for this skill.

## Procedure
1. Step one
2. Step two
```

```
## Pitfalls
- Known failure modes and fixes

## Verification
How to confirm it worked.
```

Platform-Specific Skills

Skills can restrict themselves to specific operating systems using the `platforms` field:

Value	Matches
`macos`	macOS (Darwin)
`linux`	Linux
`windows`	Windows

```
platforms: [macos]            # macOS only (e.g., iMessage, Apple
Reminders, FindMy)
platforms: [macos, linux]     # macOS and Linux
```

When set, the skill is automatically hidden from the system prompt, `skills_list()`, and slash commands on incompatible platforms. If omitted, the skill loads on all platforms.

Skill output and media delivery

When a skill response (or any agent response) includes a bare absolute path to a media file — for example `/home/user/screenshots/diagram.png` — the gateway auto-detects it, strips it from the visible text, and delivers the file natively to the user's chat (Telegram photo, Discord attachment, etc.) instead of leaving the raw path in the message.

For audio specifically, the `[[audio_as_voice]]` directive promotes audio files to native voice-message bubbles on platforms that support them (Telegram, WhatsApp).

Forcing document-style delivery: `[[as_document]]`

Sometimes you want the **opposite** of inline preview: you want the file delivered as a downloadable attachment, not a re-compressed image bubble. The classic example is a high-resolution screenshot or chart — Telegram's `sendPhoto` recompresses it to ~200 KB at 1280 px, destroying readability. A 1-2 MB PNG sent via `sendDocument` keeps the original bytes intact.

If a response (or any text inside it — typically the last line) contains the literal directive `[[as_document]]`, every media path extracted from that response is delivered as a document/file attachment rather than an image bubble:

```
Here is your rendered chart:

/home/user/.hermes/cache/chart-q4-2025.png

[[as_document]]
```

The directive is stripped before delivery, so users never see it. Granularity is intentionally all-or-nothing per response: emit `[[as_document]]` once and every image path in the same response is delivered as a document. This mirrors the scope of `[[audio_as_voice]]`.

Use it from a skill when:

- You produce screenshots or charts the user needs as files (for editing in another tool, archiving, sharing intact).
- The default lossy preview would obscure detail (small text, pixel-accurate diagrams, color-sensitive renders).

Platforms without a separate document path (e.g. SMS) fall back to whatever attachment mechanism they have.

Conditional Activation (Fallback Skills)

Skills can automatically show or hide themselves based on which tools are available in the current session. This is most useful for **fallback skills** — free or local alternatives that should only appear when a premium tool is unavailable.

```yaml
metadata:
  hermes:
    fallback_for_toolsets: [web]      # Show ONLY when these toolsets are
unavailable
```

```
    requires_toolsets: [terminal]     # Show ONLY when these toolsets are
available
    fallback_for_tools: [web_search]  # Show ONLY when these specific tools
are unavailable
    requires_tools: [terminal]        # Show ONLY when these specific tools
are available
```

Field	Behavior
`fallback_for_toolsets`	Skill is **hidden** when the listed toolsets are available. Shown when they're missing.
`fallback_for_tools`	Same, but checks individual tools instead of toolsets.
`requires_toolsets`	Skill is **hidden** when the listed toolsets are unavailable. Shown when they're present.
`requires_tools`	Same, but checks individual tools.

Example: The built-in `duckduckgo-search` skill uses `fallback_for_toolsets: [web]`. When you have `FIRECRAWL_API_KEY` set, the web toolset is available and the agent uses `web_search` — the DuckDuckGo skill stays hidden. If the API key is missing, the web toolset is unavailable and the DuckDuckGo skill automatically appears as a fallback.

Skills without any conditional fields behave exactly as before — they're always shown.

Secure Setup on Load

Skills can declare required environment variables without disappearing from discovery:

```
required_environment_variables:
  - name: TENOR_API_KEY
    prompt: Tenor API key
    help: Get a key from https://developers.google.com/tenor
    required_for: full functionality
```

When a missing value is encountered, Hermes asks for it securely only when the skill is actually loaded in the local CLI. You can skip setup and keep using the skill. Messaging

surfaces never ask for secrets in chat — they tell you to use `hermes setup` or `~/.hermes/.env` locally instead.

Once set, declared env vars are **automatically passed through** to `execute_code` and `terminal` sandboxes — the skill's scripts can use `$TENOR_API_KEY` directly. For non-skill env vars, use the `terminal.env_passthrough` config option. See Environment Variable Passthrough for details.

Skill Config Settings

Skills can also declare non-secret config settings (paths, preferences) stored in `config.yaml`:

```yaml
metadata:
  hermes:
    config:
      - key: myplugin.path
        description: Path to the plugin data directory
        default: "~/myplugin-data"
        prompt: Plugin data directory path
```

Settings are stored under `skills.config` in your config.yaml. `hermes config migrate` prompts for unconfigured settings, and `hermes config show` displays them. When a skill loads, its resolved config values are injected into the context so the agent knows the configured values automatically.

See Skill Settings and Creating Skills — Config Settings for details.

Skill Directory Structure

```
~/.hermes/skills/                  # Single source of truth
├── mlops/                         # Category directory
│   ├── axolotl/
│   │   ├── SKILL.md               # Main instructions (required)
│   │   ├── references/            # Additional docs
│   │   ├── templates/             # Output formats
│   │   ├── scripts/               # Helper scripts callable from the skill
│   │   ├── examples/              # Referenced example outputs
│   │   └── assets/                # Supplementary files
│   └── vllm/
│       └── SKILL.md
├── devops/
```

```
│   └── deploy-k8s/                # Agent-created skill
│       ├── SKILL.md
│       └── references/
├── .hub/                          # Skills Hub state
│   ├── lock.json
│   ├── quarantine/
│   └── audit.log
└── .bundled_manifest              # Tracks seeded bundled skills
```

Third-party URL and GitHub installs include `SKILL.md` plus the exact local files it references under `references/`, `templates/`, `scripts/`, `assets/`, and `examples/`. Unreferenced repository files are not copied. Hermes scans the complete quarantined bundle and records the source URL, exact content hash, scanner version, findings, timestamp, and fresh-or-cached status in `skills/.hub/lock.json`.

External Skill Directories

If you maintain skills outside of Hermes — for example, a shared `~/.agents/skills/` directory used by multiple AI tools — you can tell Hermes to scan those directories too.

Add `external_dirs` under the `skills` section in `~/.hermes/config.yaml`:

```yaml
skills:
  external_dirs:
    - ~/.agents/skills
    - /home/shared/team-skills
    - ${SKILLS_REPO}/skills
```

Paths support `~` expansion and `${VAR}` environment variable substitution.

How it works

- **Create locally, update in place**: New agent-created skills are written to `~/.hermes/skills/`. Existing skills are modified where they are found, including skills under `external_dirs`, when the agent uses `skill_manage` actions such as `patch`, `edit`, `write_file`, `remove_file`, or `delete`.
- **External dirs are not a write-protection boundary**: If an external skill directory is writable by the Hermes process, agent-managed skill updates can change files in that directory. Use filesystem permissions or a separate profile/toolset setup if shared external skills must stay read-only.

- **Local precedence**: If the same skill name exists in both the local dir and an external dir, the local version wins.
- **Full integration**: External skills appear in the system prompt index, `skills_list`, `skill_view`, and as `/skill-name` slash commands — no different from local skills.
- **Non-existent paths are silently skipped**: If a configured directory doesn't exist, Hermes ignores it without errors. Useful for optional shared directories that may not be present on every machine.

Example

```
~/.hermes/skills/               # Local (primary, read-write)
├── devops/deploy-k8s/
│   └── SKILL.md
└── mlops/axolotl/
    └── SKILL.md

~/.agents/skills/               # External (shared, mutable if writable)
├── my-custom-workflow/
│   └── SKILL.md
└── team-conventions/
    └── SKILL.md
```

All four skills appear in your skill index. If you create a new skill called `my-custom-workflow` locally, it shadows the external version.

Skill Bundles

Skill bundles are tiny YAML files that group several skills under a single slash command. When you run `/<bundle-name>`, every skill listed in the bundle loads at once — useful when a particular task always benefits from the same set of skills together.

Quick example

```
# Create a bundle for backend feature work
hermes bundles create backend-dev \
  --skill github-code-review \
  --skill test-driven-development \
  --skill github-pr-workflow \
  -d "Backend feature work — review, test, PR workflow"
```

Then in the CLI or any gateway platform:

```
/backend-dev refactor the auth middleware
```

The agent receives all three skills loaded into one user message, with any text after the slash command attached as a user instruction.

YAML schema

Bundles live in `~/.hermes/skill-bundles/<slug>.yaml` and look like this:

```yaml
name: backend-dev
description: Backend feature work — review, test, PR workflow.
skills:
  - github-code-review
  - test-driven-development
  - github-pr-workflow
instruction: |
  Always start by writing failing tests, then implement.
  Open the PR through the standard workflow with co-author tags.
```

Fields:

- `name` (optional — defaults to the filename stem) — the bundle's display name. Normalized to a hyphen slug for the slash command (`Backend Dev` → `/backend-dev`).
- `description` (optional) — short text shown in `/bundles` and `hermes bundles list`.
- `skills` (required, non-empty list) — skill names or paths relative to your skills directory. Use the same identifier you'd pass to `/<skill-name>`.
- `instruction` (optional) — extra guidance prepended to the loaded skill content. Useful for codifying "how we always use these together."

Managing bundles

```
# List all installed bundles
hermes bundles list

# Inspect one bundle
hermes bundles show backend-dev

# Create a bundle interactively (omit --skill flags to enter them one per
line)
```

```
hermes bundles create research

# Overwrite an existing bundle
hermes bundles create backend-dev --skill ... --force

# Delete a bundle
hermes bundles delete backend-dev

# Re-scan ~/.hermes/skill-bundles/ and report changes
hermes bundles reload
```

From inside a chat session, `/bundles` lists every installed bundle and its skills.

Behavior

- **Bundles take precedence over individual skills** when slugs collide. If you name a bundle `research` and you also have a skill called `research`, `/research` invokes the bundle. This is intentional — you opted into the bundle by naming it.
- **Missing skills are skipped, not fatal.** If a bundle lists `skill-foo` and you haven't installed it, the bundle still loads the skills that do resolve, and the agent gets a note listing what was skipped.
- **Bundles work in every surface** — interactive CLI, TUI, dashboard chat, and every gateway platform (Telegram, Discord, Slack, ...) — because dispatch is centralized in the same place as individual skill commands.
- **Bundles do not invalidate the prompt cache.** They generate a fresh user message at invocation time, the same way `/<skill-name>` does — no system prompt mutation.

When bundles beat installing each skill manually

Use a bundle when:

- You always pair the same skills for a recurring task (`/backend-dev`, `/release-prep`, `/incident-response`).
- You want a one-character-shorter mental model than typing several `/skill` invocations in a row.
- You want to ship a team-wide "task profile" by checking the bundle YAML into a shared dotfiles repo and symlinking it into `~/.hermes/skill-bundles/`.

A bundle is just a YAML alias — it doesn't install skills for you. The skills themselves must already be present (in `~/.hermes/skills/` or an external skill directory). Otherwise the bundle invocation just skips the missing ones.

Agent-Managed Skills (skill_manage tool)

The agent can create, update, and delete its own skills via the `skill_manage` tool. This is the agent's **procedural memory** — when it figures out a non-trivial workflow, it saves the approach as a skill for future reuse.

Skills and memory work together in the self-improvement loop: memory stores small durable facts that should always be in context, while skills store longer procedures that should load only when relevant. The background review can suggest or stage skill changes after a session, but the write-approval gate below lets you require human review before those changes land.

When the Agent Creates Skills

- After completing a complex task (5+ tool calls) successfully
- When it hit errors or dead ends and found the working path
- When the user corrected its approach
- When it discovered a non-trivial workflow

Actions

Action	Use for	Key params
`create`	New skill from scratch	`name`, `content` (full SKILL.md), optional `category`
`patch`	Targeted fixes (preferred)	`name`, `old_string`, `new_string`
`edit`	Major structural rewrites	`name`, `content` (full SKILL.md replacement)
`delete`	Remove a skill entirely	`name`
`write_file`	Add/update supporting files	`name`, `file_path`, `file_content`

Action	Use for	Key params
`remove_file`	Remove a supporting file	`name`, `file_path`

> **TIP**
>
> The `patch` action is preferred for updates — it's more token-efficient than `edit` because only the changed text appears in the tool call.

Gating agent skill writes (`skills.write_approval`)

By default the agent writes skills freely — including from the background self-improvement review that runs after a turn. If you'd rather approve every skill write first (small models that misjudge what they learned, secure environments, or just wanting eyes on the self-improvement loop), turn on the write-approval gate:

```
skills:
  write_approval: false     # false = write freely (default) | true =
require approval
```

When `write_approval: true`, every `skill_manage` write (create / edit / patch / delete / write_file / remove_file) is **staged** instead of committed — a SKILL.md is too large to review inline, so staging applies regardless of whether the write came from a foreground turn or the background review. Staged writes survive restarts under `~/.hermes/pending/skills/` and are reviewed with the same familiar approve/deny flow as dangerous commands:

```
/skills pending             # list staged skill writes + a one-line gist each
/skills diff <id>           # full unified diff (best viewed in CLI or dashboa
/skills approve <id>        # apply it (or 'all')
/skills reject <id>         # drop it (or 'all')
/skills approval on         # turn the gate on (or 'off') and persist it
```

The review surface works in the interactive CLI and on messaging platforms (diff output is truncated for chat bubbles — read the full diff on the CLI or in the pending JSON file). Memory writes have the same gate under `memory.write_approval` — see Controlling memory writes.

> The separate `skills.guard_agent_created` setting is a content scanner (dangerous-pattern heuristics), not an approval gate — the two are independent. See Guard on agent-created skill writes.

Skills Hub

Browse, search, install, and manage skills from online registries, `skills.sh`, direct well-known skill endpoints, and official optional skills.

Common commands

```
hermes skills browse                              # Browse all hub skills (official first)
hermes skills browse --source official            # Browse only official optional skills
hermes skills search kubernetes                   # Search all sources
hermes skills search react --source skills-sh     # Search the skills.sh directory
hermes skills search https://mintlify.com/docs --source well-known
hermes skills inspect openai/skills/k8s           # Preview before installing
hermes skills install openai/skills/k8s           # Install with security scan
hermes skills install official/security/1password
hermes skills install skills-sh/vercel-labs/json-render/json-render-react --force
hermes skills install well-known:https://mintlify.com/docs/.well-known/skills/mintlify
hermes skills install https://sharethis.chat/SKILL.md             # Direct URL (+ referenced support files)
hermes skills install https://example.com/SKILL.md --name my-skill # Override name when frontmatter has none
hermes skills list --source hub                   # List hub-installed skills
hermes skills check                               # Check installed hub skills for upstream updates
hermes skills update                              # Reinstall hub skills with upstream changes when needed
hermes skills audit                               # Re-scan all hub skills for security
hermes skills uninstall k8s                       # Remove a hub skill
hermes skills reset google-workspace              # Un-stick a bundled skill from "user-modified" (see below)
hermes skills reset google-workspace --restore    # Also restore the bundled version, deleting your local edits
```

```
hermes skills publish skills/my-skill --to github --repo owner/repo
hermes skills snapshot export setup.json          # Export skill config
hermes skills tap add myorg/skills-repo           # Add a custom GitHub
source
```

Supported hub sources

Source	Example	Notes
`official`	`official/security/1password`	Optional skills shipped with Hermes.
`skills-sh`	`skills-sh/vercel-labs/agent-skills/vercel-react-best-practices`	Searchable via `hermes skills search <query> --source skills-sh`. Hermes resolves alias-style skills when the skills.sh slug differs from the repo folder.
`well-known`	`well-known:https://mintlify.com/docs/.well-known/skills/mintlify`	Skills served directly from `/.well-known/skills/index.json` on a website. Search using the site or docs URL.
`url`	`https://sharethis.chat/SKILL.md`	Direct HTTP(S) URL to `SKILL.md` plus explicitly referenced support files. Name resolution: frontmatter → URL slug → interactive prompt → `--name` flag.
`github`	`openai/skills/k8s`	Direct GitHub repo/path installs and custom taps.
`clawhub`, `lobehub`,	Source-specific identifiers	Community or marketplace integrations.

Source	Example	Notes
`browse-sh`		

Integrated hubs and registries

Hermes currently integrates with these skills ecosystems and discovery sources:

1. Official optional skills (`official`)

These are maintained in the Hermes repository itself and install with built-in trust.

- Catalog: Official Optional Skills Catalog
- Source in repo: `optional-skills/`
- Example:

```
hermes skills browse --source official
hermes skills install official/security/1password
```

2. skills.sh (`skills-sh`)

This is Vercel's public skills directory. Hermes can search it directly, inspect skill detail pages, resolve alias-style slugs, and install from the underlying source repo.

- Directory: skills.sh
- CLI/tooling repo: vercel-labs/skills
- Official Vercel skills repo: vercel-labs/agent-skills
- Example:

```
hermes skills search react --source skills-sh
hermes skills inspect skills-sh/vercel-labs/json-render/json-render-react
hermes skills install skills-sh/vercel-labs/json-render/json-render-react --force
```

3. Well-known skill endpoints (`well-known`)

This is URL-based discovery from sites that publish `/.well-known/skills/index.json`. It is not a single centralized hub — it is a web discovery convention.

- Example live endpoint: Mintlify docs skills index
- Reference server implementation: vercel-labs/skills-handler
- Example:

```
hermes skills search https://mintlify.com/docs --source well-known
hermes skills inspect well-known:https://mintlify.com/docs/.well-
known/skills/mintlify
hermes skills install well-known:https://mintlify.com/docs/.well-
known/skills/mintlify
```

4. Direct GitHub skills (`github`)

Hermes can install directly from GitHub repositories and GitHub-based taps. This is useful when you already know the repo/path or want to add your own custom source repo.

Default taps (browsable without any setup):

- openai/skills
- anthropics/skills
- huggingface/skills
- NVIDIA/skills — NVIDIA-verified skills (signed `skill.oms.sig` + governance `skill-card.md`)
- garrytan/gstack
- Example:

```
hermes skills install openai/skills/k8s
hermes skills tap add myorg/skills-repo
```

Category groupings (`skills.sh.json`). A GitHub tap may ship a `skills.sh.json` file at its repo root following the skills.sh schema. Its `groupings` (each with a `title` and a list of skill names) are read at index time and become the category labels shown in the Skills Hub page — instead of a tag-derived guess. This is generic: any tap that ships the file gets real categorization, no Hermes-side changes required.

```
{
  "$schema": "https://skills.sh/schemas/skills.sh.schema.json",
```

```
  "groupings": [
    { "title": "Inference AI", "skills": ["dynamo-recipe-runner", "dynamo-
router-sla"] },
    { "title": "Decision Optimization", "skills": ["cuopt-developer",
"cuopt-install"] }
  ]
}
```

5. ClawHub (`clawhub`)

A third-party skills marketplace integrated as a community source.

- Site: clawhub.ai
- Hermes source id: `clawhub`

6. LobeHub (`lobehub`)

Hermes can search and convert agent entries from LobeHub's public catalog into installable Hermes skills.

- Site: LobeHub
- Public agents index: chat-agents.lobehub.com
- Backing repo: lobehub/lobe-chat-agents
- Hermes source id: `lobehub`

7. browse.sh (`browse-sh`)

Hermes integrates with browse.sh, Browserbase's catalog of 200+ site-specific browser-automation SKILL.md files (Airbnb, Amazon, arXiv, 12306.cn, Etsy, Xero, and many more). Each skill describes how to drive one website end-to-end and is suitable for use with Hermes' browser tools and any browser-automation skills you already have installed.

- Site: browse.sh
- Catalog API: `https://browse.sh/api/skills`
- Hermes source id: `browse-sh`
- Trust level: `community`

```
hermes skills search airbnb --source browse-sh
hermes skills inspect browse-sh/airbnb.com/search-listings-ddgioa
hermes skills install browse-sh/airbnb.com/search-listings-ddgioa
```

Identifiers use the form `browse-sh/<hostname>/<task-id>` and match the slug exposed by the browse.sh catalog. Content is resolved through the per-skill detail endpoint (`/api/skills/<slug>` → `skillMdUrl`), not through the catalog's GitHub `sourceUrl`.

8. Direct URL (`url`)

Install `SKILL.md` directly from any HTTP(S) URL — useful when an author hosts a skill on their own site (no hub listing, no GitHub path to type). Hermes also fetches explicitly referenced files under `references/`, `templates/`, `scripts/`, `assets/`, and `examples/`, then scans and installs the complete bundle.

- Hermes source id: `url`
- Identifier: the URL itself (no prefix needed)
- Scope: `SKILL.md` plus exact referenced support files in the allowlisted directories. Hermes does not enumerate or copy unrelated files from the host.

```
hermes skills install https://sharethis.chat/SKILL.md
hermes skills install https://example.com/my-skill/SKILL.md --category
productivity
```

Name resolution, in order:

1. `name:` field in the SKILL.md YAML frontmatter (recommended — every well-formed skill has one).
2. Parent directory name from the URL path (e.g. `.../my-skill/SKILL.md` → `my-skill`, or `.../my-skill.md` → `my-skill`), when it's a valid identifier (`^[a-z][a-z0-9_-]*$`).
3. Interactive prompt on a terminal with a TTY.
4. On non-interactive surfaces (the `/skills install` slash command inside the TUI, gateway platforms, scripts), a clean error pointing at the `--name` override.

```
# Frontmatter has no name and the URL slug is unhelpful — supply one:
hermes skills install https://example.com/SKILL.md --name sharethis-chat

# Or inside a chat session:
/skills install https://example.com/SKILL.md --name sharethis-chat
```

Trust level is always `community` — the same security scan runs as for every other source. The URL is stored as the install identifier, so `hermes skills update` re-fetches from the same URL automatically when you want to refresh.

Security scanning and `--force`

All hub-installed skills go through a **security scanner** that checks for data exfiltration, prompt injection, destructive commands, supply-chain signals, and other threats.

`hermes skills inspect ...` now also surfaces upstream metadata when available:

- repo URL
- skills.sh detail page URL
- install command
- weekly installs
- upstream security audit statuses
- well-known index/endpoint URLs

Use `--force` when you have reviewed a third-party skill and want to override a non-dangerous policy block:

```
hermes skills install skills-sh/anthropics/skills/pdf --force
```

Important behavior:

- `--force` can override policy blocks for caution/warn-style findings.
- `--force` does **not** override a `dangerous` scan verdict.
- Official optional skills (`official/...`) are treated as built-in trust and do not show the third-party warning panel.

Trust levels

Level	Source	Policy
`builtin`	Ships with Hermes	Always trusted
`official`	`optional-skills/` in the repo	Built-in trust, no third-party warning
`trusted`	Trusted registries/repos such as `openai/skills`, `anthropics/skills`, `huggingface/skills`, `NVIDIA/skills`	More permissive policy than community sources

Level	Source	Policy
`community`	Everything else (`skills.sh`, well-known endpoints, custom GitHub repos, most marketplaces)	Non-dangerous findings can be overridden with `--force`; `dangerous` verdicts stay blocked

Update lifecycle

The hub now tracks enough provenance to re-check upstream copies of installed skills:

```
hermes skills check          # Report which installed hub skills changed upstream
hermes skills update         # Reinstall only the skills with updates available
hermes skills update react   # Update one specific installed hub skill
```

This uses the stored source identifier plus the current upstream bundle content hash to detect drift.

> **GITHUB RATE LIMITS**
>
> Skills hub operations use the GitHub API, which has a rate limit of 60 requests/hour for unauthenticated users. If you see rate-limit errors during install or search, set `GITHUB_TOKEN` in your `.env` file to increase the limit to 5,000 requests/hour. The error message includes an actionable hint when this happens.

Publishing a custom skill tap

If you want to share a curated set of skills — for your team, your org, or publicly — you can publish them as a **tap**: a GitHub repository other Hermes users add with `hermes skills tap add <owner/repo>`. No server, no registry sign-up, no release pipeline. Just a directory of `SKILL.md` files.

Repo layout

A tap is any GitHub repo (public or private — private needs `GITHUB_TOKEN`) laid out like this:

```
owner/repo
├── skills/                        # default path; configurable per-tap
│   ├── my-workflow/
│   │   ├── SKILL.md               # required
│   │   ├── references/            # optional supporting files
│   │   ├── templates/
│   │   └── scripts/
│   ├── another-skill/
│   │   └── SKILL.md
│   └── third-skill/
│       └── SKILL.md
└── README.md                      # optional but helpful
```

Rules:

- Each skill lives in its own directory under the tap's root path (default `skills/`).
- The directory name becomes the skill's install slug.
- Each skill directory must contain a `SKILL.md` with standard SKILL.md frontmatter (`name`, `description`, plus optional `metadata.hermes.tags`, `version`, `author`, `platforms`, `metadata.hermes.config`).
- Subdirectories like `references/`, `templates/`, `scripts/`, `assets/` are downloaded alongside `SKILL.md` at install time.
- Skills whose directory name starts with `.` or `_` are ignored.

Hermes discovers skills by listing every subdirectory of the tap path and probing each for `SKILL.md`.

Minimal tap example

```
my-org/hermes-skills
└── skills/
    └── deploy-runbook/
        └── SKILL.md
```

`skills/deploy-runbook/SKILL.md`:

```yaml
---
name: deploy-runbook
description: Our deployment runbook — services, rollback, Slack channels
version: 1.0.0
author: My Org Platform Team
metadata:
  hermes:
```

```
    tags: [deployment, runbook, internal]
---

# Deploy Runbook

Step 1: ...
```

After pushing that to GitHub, any Hermes user can subscribe and install:

```
hermes skills tap add my-org/hermes-skills
hermes skills search deploy
hermes skills install my-org/hermes-skills/deploy-runbook
```

Non-default paths

If your skills don't live under `skills/` (common when you're adding a `skills/` subtree to an existing project), edit the tap entry in `~/.hermes/skills/.hub/taps.json`:

```
{
  "taps": [
    {"repo": "my-org/platform-docs", "path": "internal/skills/"}
  ]
}
```

The `hermes skills tap add` CLI defaults new taps to `path: "skills/"`; edit the file directly if you need a different path. `hermes skills tap list` shows the effective path per tap.

Installing individual skills directly (without adding a tap)

Users can also install a single skill from any public GitHub repo without adding the whole repo as a tap:

```
hermes skills install owner/repo/skills/my-workflow
```

Useful when you want to share one skill without asking the user to subscribe to your whole registry.

Trust levels for taps

New taps are assigned `community` trust by default. Skills installed from them run through the standard security scan and show the third-party warning panel on first install. If your org or a widely-trusted source should get higher trust, add its repo to `TRUSTED_REPOS` in `tools/skills_hub.py` (requires a Hermes core PR).

Tap management

```
hermes skills tap list                          # show all configured
taps
hermes skills tap add myorg/skills-repo         # add (default path:
skills/)
hermes skills tap remove myorg/skills-repo      # remove
```

Inside a running session:

```
/skills tap list
/skills tap add myorg/skills-repo
/skills tap remove myorg/skills-repo
```

Taps are stored in `~/.hermes/skills/.hub/taps.json` (created on demand).

Bundled skill updates (`hermes skills reset`)

Hermes ships with a set of bundled skills in `skills/` inside the repo. On install and on every `hermes update`, a sync pass copies those into `~/.hermes/skills/` and records a manifest at `~/.hermes/skills/.bundled_manifest` mapping each skill name to the content hash at the time it was synced (the **origin hash**).

On each sync, Hermes recomputes the hash of your local copy and compares it to the origin hash:

- **Unchanged** → safe to pull upstream changes, copy the new bundled version in, record the new origin hash.
- **Changed** → treated as **user-modified** and skipped forever, so your edits never get stomped.

The protection is good, but it has one sharp edge. If you edit a bundled skill and then later want to abandon your changes and go back to the bundled version by just copy-pasting

from `~/.hermes/hermes-agent/skills/`, the manifest still holds the *old* origin hash from whenever the last successful sync ran. Your fresh copy-paste contents (current bundled hash) won't match that stale origin hash, so sync keeps flagging it as user-modified.

`hermes skills reset` is the escape hatch:

```
# Safe: clears the manifest entry for this skill. Your current copy is preserved,
# but the next sync re-baselines against it so future updates work normally.
hermes skills reset google-workspace

# Full restore: also deletes your local copy and re-copies the current bundled
# version. Use this when you want the pristine upstream skill back.
hermes skills reset google-workspace --restore

# Non-interactive (e.g. in scripts or TUI mode) — skip the --restore confirmation.
hermes skills reset google-workspace --restore --yes
```

The same command works in chat as a slash command:

```
/skills reset google-workspace
/skills reset google-workspace --restore
```

> **PROFILES**
>
> Each profile has its own `.bundled_manifest` under its own `HERMES_HOME`, so `hermes -p coder skills reset <name>` only affects that profile.

Slash commands (inside chat)

All the same commands work with `/skills`:

```
/skills browse
/skills search react --source skills-sh
/skills search https://mintlify.com/docs --source well-known
/skills inspect skills-sh/vercel-labs/json-render/json-render-react
/skills install openai/skills/skill-creator --force
/skills check
/skills update
/skills reset google-workspace
```

```
/skills list
```

Official optional skills still use identifiers like `official/security/1password` and `official/migration/openclaw-migration`.